

Quarto Manuals

Tinashe M. Tapera^{1,2}, Michelle Audirac², Danielle Braun², and Christopher Golden¹

¹Harvard T.H. Chan School of Public Health, Harvard University

²Harvard T.H. Chan School of Public Health, Harvard University

Author Note

Tinashe M. Tapera  <https://orcid.org/0000-0001-9080-5010>

Christopher Golden  <https://orcid.org/0000-0002-2258-7493>

The authors have no conflicts of interest to disclose. Author roles were classified using the Contributor Role Taxonomy (CRediT; credit.niso.org) as follows: Tinashe M. Tapera: conceptualization, writing, software; Michelle Audirac: writing, supervision; Danielle Braun: writing, supervision; Christopher Golden: project administration, writing, supervision

Correspondence concerning this article should be addressed to Tinashe M. Tapera, Harvard T.H. Chan School of Public Health, Harvard University, 655 Huntington Ave, Boston, MA 2115, USA, Harvard T.H. Chan School of Public Health, Harvard University, 655 Huntington Ave, Boston, MA 2115, USA, Email: ttapera@hsph.harvard.edu

Abstract

See below.

Impact Statement

Scientific programming workflows are becoming more complex, while existing support mechanisms often remain poorly matched to that complexity. We see Quarto Manuals as a practical contribution to this problem and as directly aligned with the interests of the US-RSE community. Research software engineers are often responsible not only for building software, but also for making workflows understandable, reproducible, maintainable, and usable by others. Quarto Manuals are designed to support that work by combining explanation, code execution, and verification in an ordered, operator-facing format. This contribution has three main highlights. First, our work promotes technical resources for documenting and delivering rigorous, complex scientific workflows. Second, it supports the visibility and value of research software engineering by treating workflow design, execution, and handoff as first-class engineering artifacts, rather than incidental burdens or purely local practice. Third, it serves the broader community by making it easier for collaborators with different levels of programming experience to run, inspect, and adapt computational workflows in a structured and auditable way. Perhaps most importantly, Quarto Manuals are also intended to be complementary to the growing use of AI-assisted programming. As AI-generated code becomes more common, research teams will increasingly need workflow structures that make human-written instructions *and* LLM-generated code easier to inspect, test, reproduce, and maintain together. In that sense, Quarto Manuals may provide a useful [harness](#) for incorporating AI-assisted programming into scientific workflows, without sacrificing rigor or transparency. Ultimately, we hope Quarto Manuals helps make scientific programming more accessible, reproducible, and inclusive, while supporting the high standards of software engineering and collaboration that the US-RSE community promotes.

Keywords: Quarto, notebook driven development, reproducible research, scientific programming, AI

Quarto Manuals

Climate-Smart Public Health (CSPH) is a data-driven approach to detect, quantify, and adapt to climate-related health impacts at a global scale, with efforts spanning surveillance, risk assessment, early warning and response, and healthcare systems adaptation ([Golden et al., 2025](#)). In practice, CSPH depends on complex computational workflows that integrate heterogeneous climate, environmental, and health data and that must remain understandable, reproducible, and adaptable across collaborators and local settings.

Despite the central role of these workflows in academia and industry, effective, reproducible documentation remains an overlooked and under-served part of delivering high quality software ([Aghajani et al., 2019](#)). This issue is likely to become more acute as software development continues to expand in scale and complexity ([Martinelli, 2026](#)) and as the volume and sophistication of AI generated code submitted to public repositories continues to increase ([Robbes et al., 2026](#)). Developers therefore need solutions beyond traditional static documentation to support workflow execution, verification, and reproducibility, in practical and user-friendly ways.

Conventional documentation can describe a workflow, but for complex, multi-step, order-dependent tasks and projects, it is often insufficient at capturing and verifying critical runtime execution details such as platform dependencies, configuration, versions, and relationships between steps ([Ebert et al., 2015](#)). Command-line interface (CLI) wrappers can improve automation and reproducibility of a declarative workflow by providing a directly executable interface for complex tasks ([van der Aalst et al., 2009](#)). However, developing and maintaining CLI wrappers can be time-consuming and introduces a persistent trade-off between control and flexibility: highly specialized wrappers can improve consistency, but may be inflexible and difficult to maintain, while more generic wrappers enable complexity by shifting the burden to the user through large numbers of flags and configuration options ([Brack et al., 2022](#); [Sadiq et al., 2005](#)). Lastly, AI and agentic systems are promising, but their reproducibility remains limited by non-determinism ([Siddiq et al., 2025](#)). Furthermore, current practical agentic workflow

implementations (such as harnesses) still require careful human oversight to achieve acceptable reliability ([Agrawal et al., 2026](#)).

Executable literate programming approaches can encode and enforce complex workflow steps *and* generate idempotent results ([Knuth, 1992](#)).

Quarto is a modern, plain-text, multi-language, and multi-output literate programming framework for creating reproducible documents and workflows ([Allaire et al., 2026](#)). Building on Quarto with a carefully designed set of templates and extensions, we introduce **Quarto Manuals**: a framework for **executable software manuals** that guide users through workflows as ordered, interactive pages that combine explanation, code execution, and step verification. Rather than hiding workflow decisions and steps, Quarto Manuals keep them visible and editable, making them ideal for research computing contexts where local constraints and expert judgment often shape execution parameters. The framework is powered by `quarto-emit`, a lightweight backend extension that materializes conventional workflow artifacts from manual pages when needed. We demonstrate the approach with three examples of increasing sophistication: a simple manual for staging datasets in your project space; an intermediate manual for geospatial aggregation of environmental exposure data; and a complex manual for creating robust, reproducible, fully containerized geospatial data science environments on High Performance Computing (HPC) clusters. All examples are available at the project [website](#).

For authors, Quarto Manuals provide a structured yet adaptable way to turn recurring workflows into reusable operators' manuals for themselves and colleagues; for operators, they provide a stepwise, testable process for generating reproducible setup and execution artifacts and confirming progress throughout execution in a structured and auditable manner.

Quarto Manuals promise to improve the reproducibility and reliability of scientific workflows — particularly in contexts like CSPH where local constraints and expert judgment shape execution parameters and conditions — while also providing an accessible and interactive experience for users. In future work, we hope to explore how Quarto Manuals may be used in conjunction with the increasing adoption of AI and agentic systems in software development to

provide a framework for ensuring that computational workflows remain transparent, verifiable, and reproducible in the face of increasing complexity and non-determinism.

References

- Aghajani, E., Nagy, C., Vega-Márquez, O. L., Linares-Vásquez, M., Moreno, L., Bavota, G., & Lanza, M. (2019). Software Documentation Issues Unveiled. *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, 1199–1210. <https://doi.org/10.1109/ICSE.2019.00122>
- Agrawal, S., Anadkat, H. B., Athimoolam, K. K., Bhardwaj, H., Chowdhury, T., Gao, S., Kamat, P. K., Makwana, V., Shariff, M. H., Badkul, A., Xie, L., & Sinitskiy, A. V. (2026, January 6). *Can AI Conduct Autonomous Scientific Research? Case Studies on Two Real-World Tasks*. <https://doi.org/10.64898/2026.01.05.697809> (Pre-published)
- Allaire, J. J., Teague, C., Scheidegger, C., Xie, Y., Dervieux, C., & Woodhull, G. (2026). *Quarto* (Version 1.10) [Computer software]. <https://doi.org/10.5281/zenodo.5960048>
- Brack, P., Crowther, P., Soiland-Reyes, S., Owen, S., Lowe, D., Williams, A. R., Groom, Q., Dillen, M., Coppens, F., Grüning, B., Eguinoa, I., Ewels, P., & Goble, C. (2022). Ten simple rules for making a software tool workflow-ready. *PLOS Computational Biology*, 18(3), e1009823. <https://doi.org/10.1371/journal.pcbi.1009823>
- Ebert, P., Müller, F., Nordström, K., Lengauer, T., & Schulz, M. H. (2015). A general concept for consistent documentation of computational analyses. *Database: The Journal of Biological Databases and Curation*, 2015, bav050. <https://doi.org/10.1093/database/bav050>
- Golden, C. D., Childs, M. L., Mudele, O. E., Andriamizarasoa, F. A., Bouley, T. A., Nicola, G. D., Fontaine, M. A., Huybers, P. J., Mahatante, P. T., Rabemananjara, R., Rakotoarison, N., Ramambason, H. R., Ramihantaniarivo, H., Randriamady, H. J., Randriatsara, H., Ravelomanantsoa, M. A., Razafinimanana, A. K. S., Rigden, A. J., Shumake-Guillemot, J., ... Dominici, F. (2025). Climate-smart public health for global health resilience. *The Lancet Planetary Health*, 9(8). <https://doi.org/10.1016/j.lanplh.2025.101293>
- Knuth, D. E. (1992). *Literate Programming*. Center for the Study of Language and Information.

<https://press.uchicago.edu/ucp/books/book/distributed/L/bo3629677.html>

Martinelli, N. (2026, January 16). *Software Heritage Activity Report: 2025*. Software Heritage.

<https://www.softwareheritage.org/2026/01/16/software-heritage-activity-report-2025/>

Robbes, R., Matricon, T., Degueule, T., Hora, A., & Zacchiroli, S. (2026). Agentic Much?

Adoption of Coding Agents on GitHub. *ACM Transactions on Software Engineering and Methodology*. <https://doi.org/10.1145/3822180>

Sadiq, S. W., Orlowska, M. E., & Sadiq, W. (2005). Specification and validation of process constraints for flexible workflows. *Information Systems*, 30(5), 349–378.

<https://doi.org/10.1016/j.is.2004.05.002>

Siddiq, M. L., Islam-Gomes, A., Sekerak, N., & Santos, J. C. S. (2025, November 29). *Large Language Models for Software Engineering: A Reproducibility Crisis*.

<https://doi.org/10.48550/arXiv.2512.00651>

van der Aalst, W. M. P., Pesic, M., & Schonenberg, H. (2009). Declarative workflows: Balancing between flexibility and support. *Computer Science - Research and Development*, 23(2), 99–113. <https://doi.org/10.1007/s00450-009-0057-9>